CIS 4020 · DATA STRUCTURES

Hash Tables

Calculate the address instead of searching for it

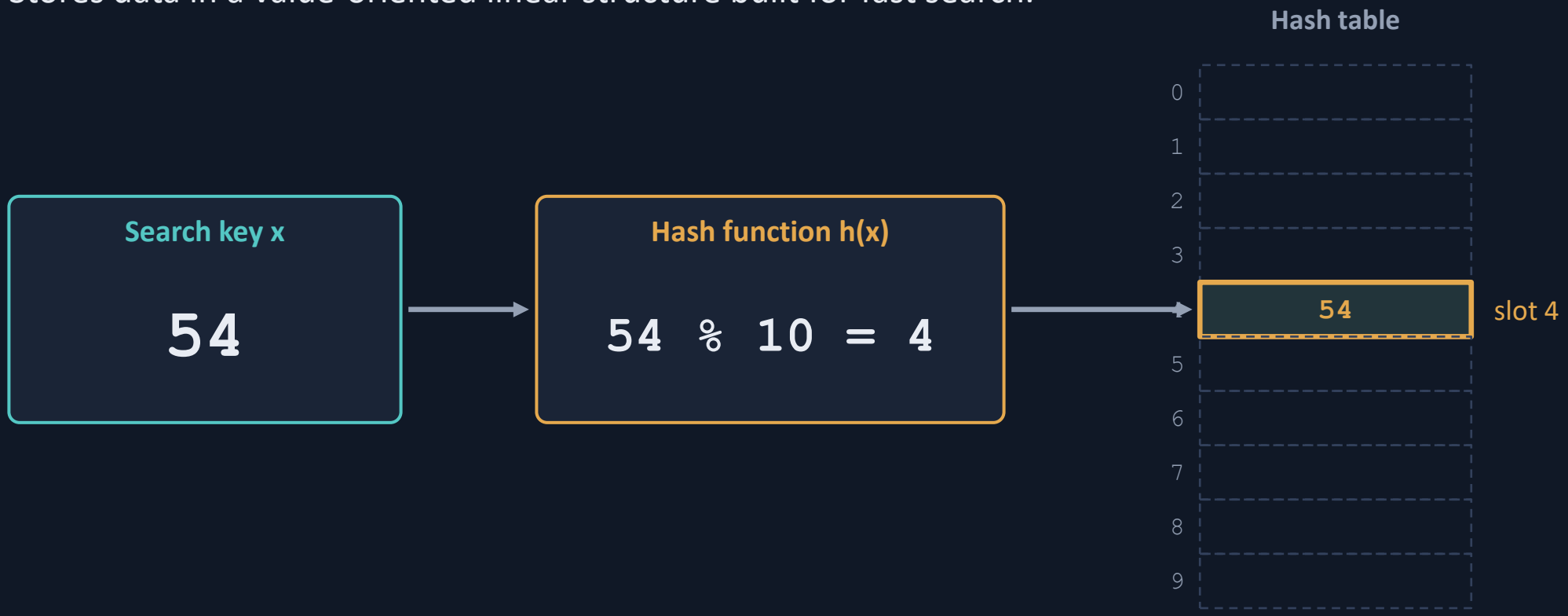


Walls and Mirrors

PART 1 OF 3 · THE IDEA

What is a hash table?

Stores data in a value-oriented linear structure built for fast search.



The key 54 passes through h(x) = x % 10 and lands directly in slot 4 of a 10-slot table. No other slot is checked.

TERMINOLOGY

Three terms, one goal

Hashing

Converting the search key x into its hash value.

Hash function h(x)

The address calculator. Turns a key into a table index.

Hash table

The data structure that stores the items, usually an array.

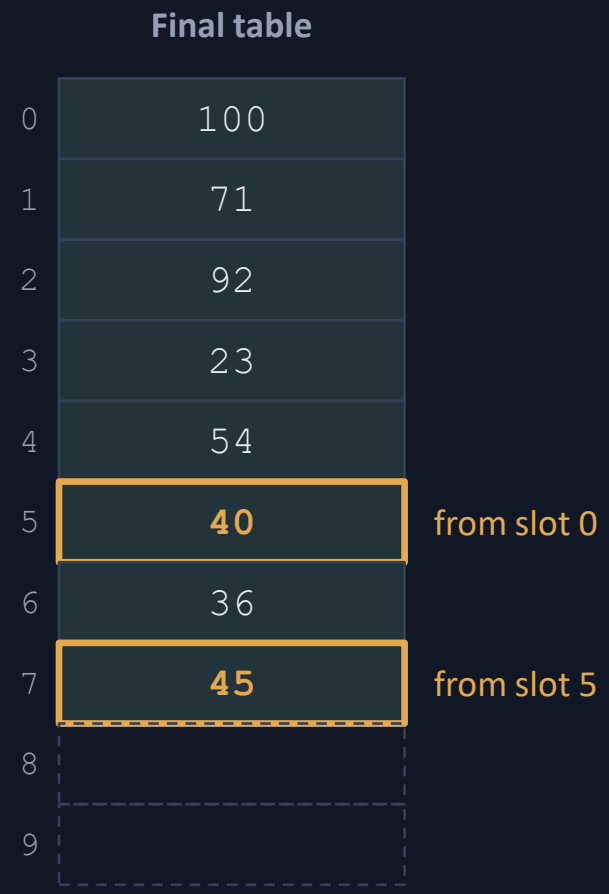
O(1)

average time for search, insert, and delete, as long as collisions stay rare

You do not have to search as many locations for an item. Ideally, you check exactly one.

h(x) = x % 10

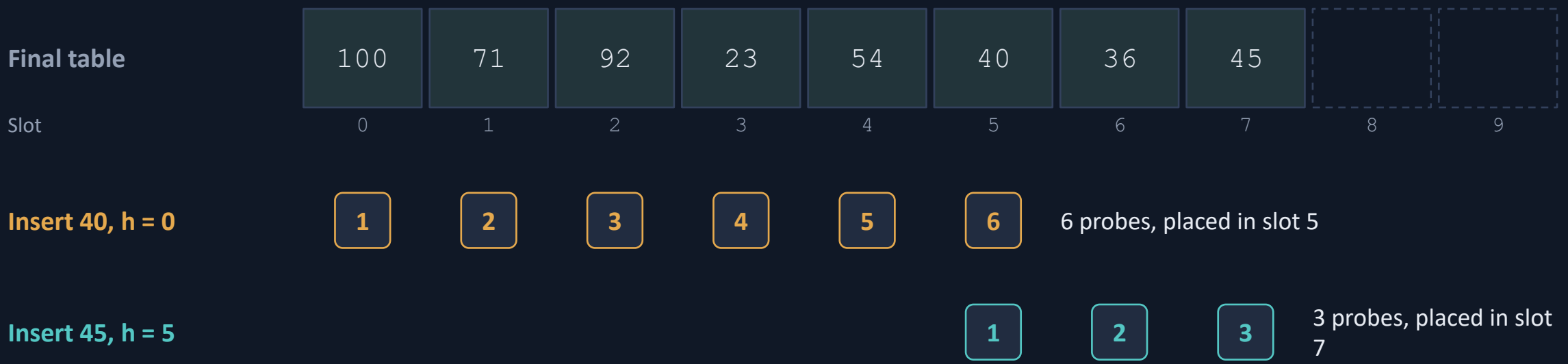
Key x	h(x)	Final slot	Probes
100	0	0	1
92	2	2	1
54	4	4	1
71	1	1	1
23	3	3	1
36	6	6	1
40	**0**	**5**	**6**
45	**5**	**7**	**3**



Six keys land in their home slot with one probe. 40 and 45 collide and are shown in bold gold: 40 moves from slot 0 to slot 5, and 45 moves from slot 5 to slot 7.

WALKING THE PROBES

Collisions create collisions



40 landed in slot 5, which is the home slot of 45. One collision caused the next. Runs of full slots like this are called primary clustering.

Numbered squares show each probe in order. Every check counts, including the first one.

What makes a good hash function

Fast to compute

Computed on every operation, so it must be O(1). That is easy to achieve.

Spreads keys evenly

Distributes items uniformly across the table to avoid collisions. Think about whether your keys are random or patterned.

Uses the whole key

If h ignores part of the key, keys that differ only in that part always collide.

Perfect hash function

Produces no collisions at all. Only possible when every search key is known in advance.

A collision happens when h(x) maps more than one item to the same location.

INTEGER TECHNIQUES

Turning integers into table slots

Key: 001364825

Selecting digits

Take the 4th digit and the last digit: 3 and 5.

h = 35

Folding: add the digits

0 + 0 + 1 + 3 + 6 + 4 + 8 + 2 + 5

h = 29

Folding: add groups of three

001 + 364 + 825

h = 1190, too big for a 101-slot table

Modulus arithmetic

001364825 % 101, using a prime table size

h = 12

Folding can overflow the table, so it is often followed by a modulus. Modulus functions should use a prime table size.

STRING KEYS

Strings become integers first

ASCII values

Use each character's ASCII (or Unicode) code.

Alphabet position

A = 1, B = 2, and so on through Z = 26.

Binary positions

Write each position in 5 bits and concatenate. N = 14 = 01110.

```
C = 3   ->  00011
A = 1   ->  00001
T = 20  ->  10100

000110000110100 = 3124

h(CAT) = 3124 % 101 = 94
```

Just adding the letter values is a weak choice: CAT, ACT, and TAC all add up to 24 and collide.

COLLISION RESOLUTION

Open addressing: probe for an open slot

Linear probing

Probe i checks $h(x) + i$

Offsets: 0, 1, 2, 3, 4, ...

Primary clustering: runs of full slots build up near the home slots and keep growing.

Quadratic probing

Probe i checks $h(x) + i^2$

Offsets: 0, 1, 4, 9, 16, ...

Secondary clustering: keys with the same home slot still follow the same path.

Double hashing

Probe i checks $h_1(x) + i \cdot h_2(x)$

The step size depends on the key.

Largely avoids both kinds of clustering when h_2 is never 0 and the table size is prime.

All slot numbers are taken modulo the table size. A search follows the same sequence and stops at the key or at an empty slot.

DOUBLE HASHING

Double hashing, step by step

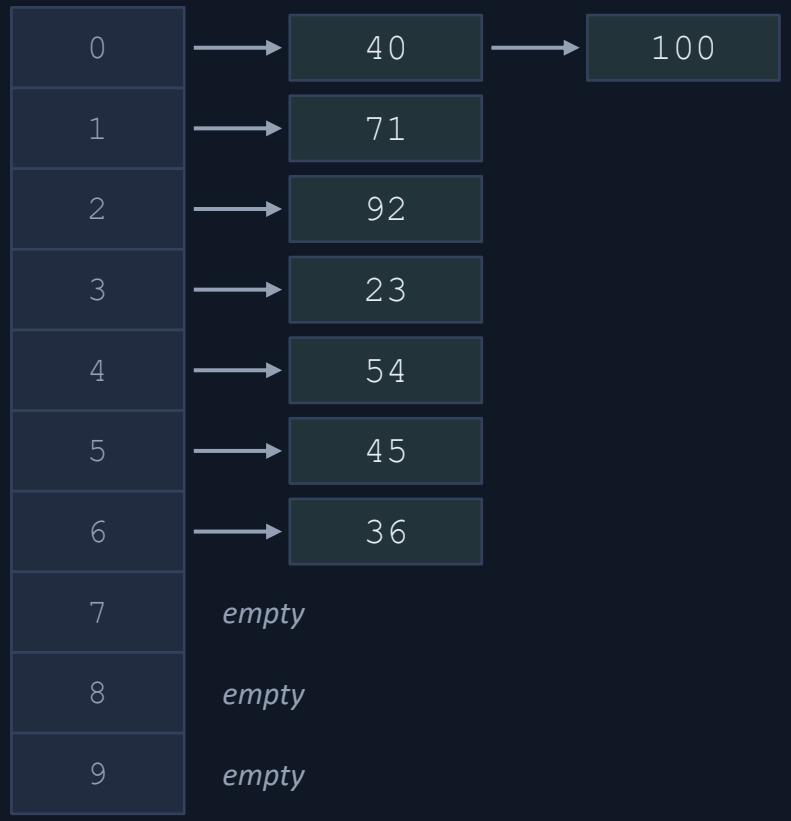



```
h1(x) = x % 11          h2(x) = 7 - (x % 7)
58: h1 = 3                                          -> slot 3
14: h1 = 3 taken, h2 = 7: try 10                    -> slot 10
91: h1 = 3 taken, h2 = 7: try 10 taken, try 6       -> slot 6
25: h1 = 3 taken, h2 = 3: try 6 taken, try 9        -> slot 9
```

14, 91, and 25 all start at slot 3, but 25 steps by 3 instead of 7, so it does not follow the others.

COLLISION RESOLUTION

Separate chaining: buckets



Why chaining behaves differently

A collision adds one more node to that slot's list.

No probing into other slots, so no clustering.

Deleting is an ordinary linked list removal.

The table can hold more items than slots, so the load factor can exceed 1.

The cost: memory for the links, and a long chain is as slow as any list.

Board example keys stored with chaining. Bucket 0 holds 40 and then 100, since 40 arrived second and was inserted at the front. Buckets 7 to 9 are empty.

EFFICIENCY

The load factor decides the cost

α = items in the table / table size

Our board example

8 items in 10 slots

$\alpha = 8 / 10 = 0.8$, above the 2/3 goal

Separate chaining

α is the average chain length, and it can exceed 1.

2/3

Goal for open addressing: keep α from exceeding two thirds

The cost depends on how full the table is, not on how many items it holds.

EFFICIENCY

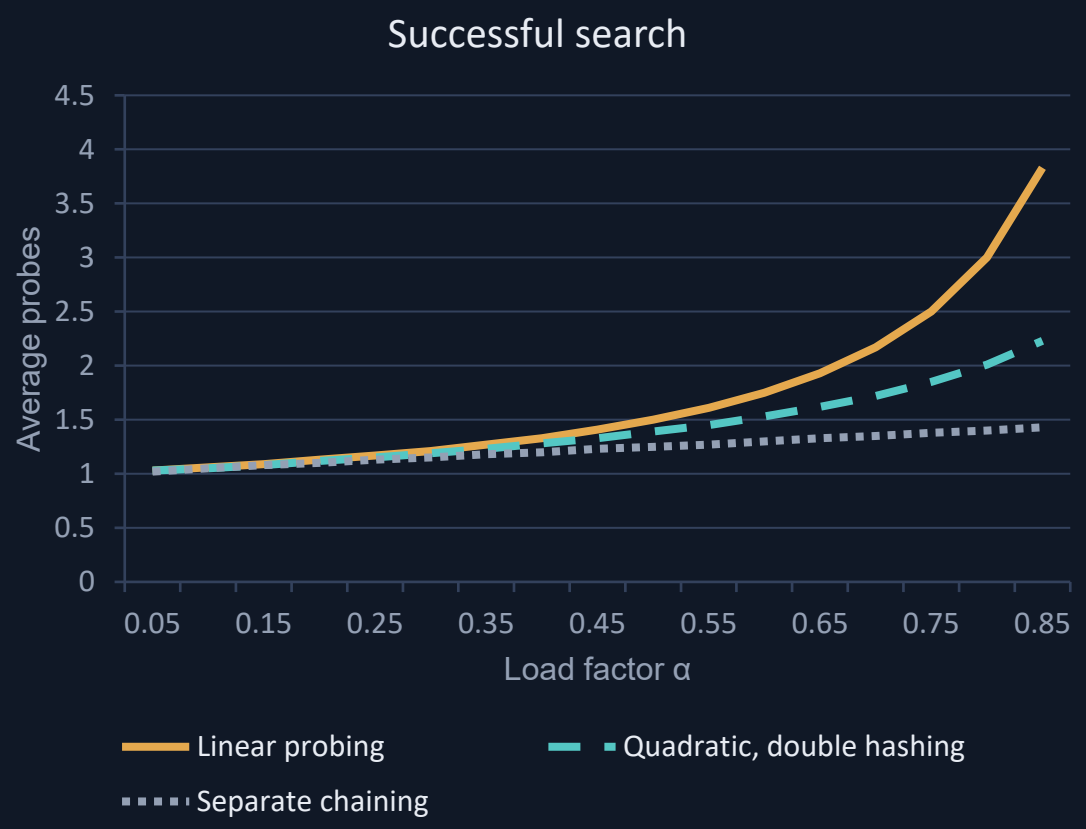
Average probes per search

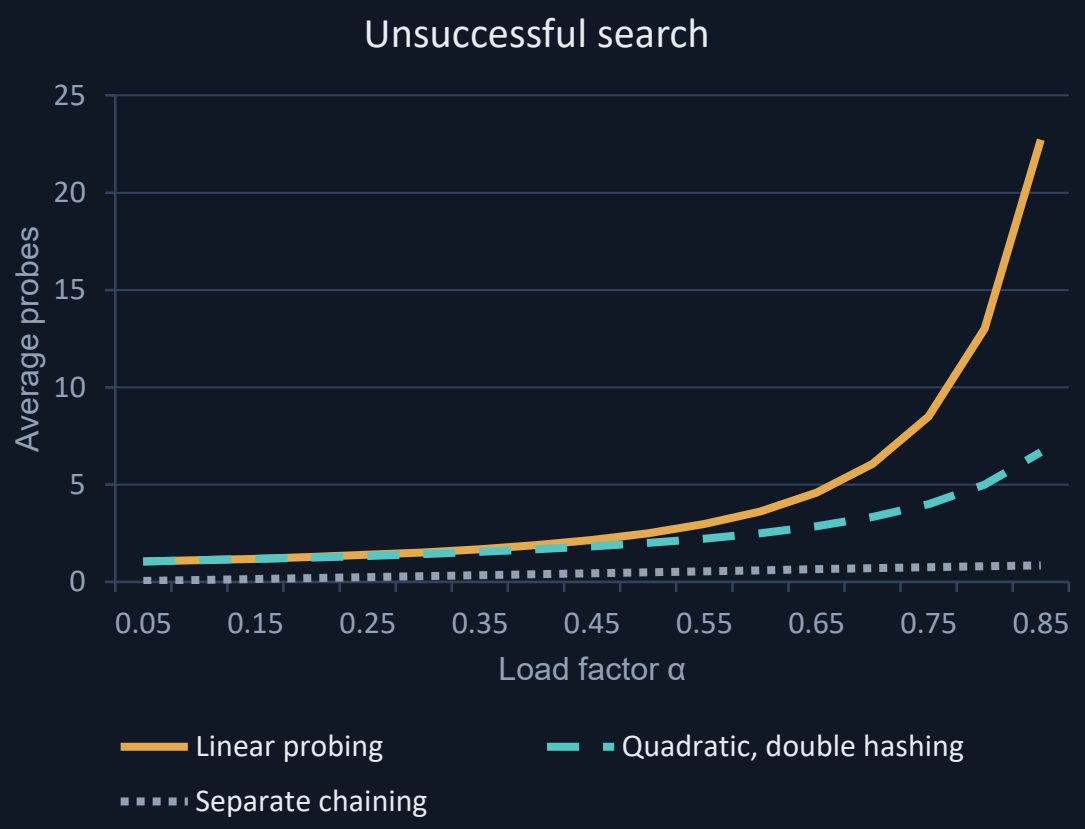
Technique	Successful search	Unsuccessful search
Linear probing	$1/2\ [1 + 1/(1 - \alpha)]$	$1/2\ [1 + 1/(1 - \alpha)^2]$
Quadratic probing, double hashing	$-\ln(1 - \alpha) / \alpha$	$1 / (1 - \alpha)$
Separate chaining	$1 + \alpha/2$	α

At $\alpha = 0.75$, linear probing expects 2.5 probes to find a key but 8.5 to discover that a key is missing.

EFFICIENCY

How fast the cost grows





Solid line: linear probing. Dashed: quadratic probing and double hashing. Dotted: separate chaining. Values come from the formulas on the previous slide.

WRAP UP

What to remember

1. h(x) calculates where an item belongs, so most operations are O(1) on average.
2. Collisions are resolved by probing (linear, quadratic, double hashing) or by chaining.
3. Cost depends on the load factor α. Keep α at or below 2/3 for open addressing.
4. Use a prime table size, and always cap the number of probes in code.

Reading: Java (CIS) Section 13.2, Hashing. C/C++ (CS) Chapter 24, Hashing as a Map Implementation.